SOAP API Development Best Practices

Edition 2

February 2018

Disclaimer

This documentation is the confidential and proprietary information of Navitaire LLC and may be used, modified, altered, copied, reproduced, or transferred only in accordance with a written license agreement executed by Navitaire.

Table of Contents

Introduction

Purpose

This document describes the recommended approach to API development using the New Skies SOAP web services.

Other product information available for the SOAP API Web Services API includes the following:

SOAP API Web Services Programming Reference Guide

This online help file (.chm) provides a brief description for the methods supported in the AccountManager, AgentManager, AllotmentManager, BookingManager, ContentManager, ExternalMessageManager, OperationsManager, PersonManager, ProcessManager, QueueManager,ScheduleManager, SessionManager, SubscriptionManager, TravelCommerceManager, UtilitiesManager, and VoucherManager. Each method is linked to a request and response SOAP example for your use.

ObjectReference.chm

This inline documentation is a generated HTML online help file (.chm) supplied within the code. It includes detailed information on each of the methods supported in the manager clients. The field names for each method include details on which fields are required, conditionally required, optional, or not used. Exceptions returned by the API are documented here in the fault contract namespace.

SOAP API Web Services Functional Overview

This document, available as a PDF, provides a high-level view of the functionality supported in this product. The audience for this document would use this information to determine if the scope of the Web Services APIs fits their organization requirements.

SOAP API Web Services Method and Contract Changes

This document compares method and contract changes between releases. It includes new as well as deprecated methods for each client manager. Contract changes are also included.

Performance

Transactions Per Minute (TPM)

Navitaire makes no recommendations regarding the number of calls per minute for any given method. TPM performance impact is highly data-dependent; primarily with regard to:

- Number of flights and fares in the market
- Whether sum-of-sectors faring is enabled
- Whether all-up pricing is enabled (number and complexity of the taxes/fees that must be considered)
- Load factor of each request (number of requested days, etc.)
- Available server capacity
- Number of concurrent users.

Session Management

Since New Skies requires a session for all API interactions, proper session management is paramount to performance. Applications should re-use sessions for repetitive functions and explicitly call **Logout** at the end of processes to dispose of the session and free app server memory.

Logon/Logout

New Skies requires that all consumers of the API logon to validate credentials, verify access level, and to instantiate the session that will be used throughout the subsequent interactions. The Logon method may seem lightweight, but in reality a series of lookups is happening in the background. This makes the Logon call one of the heavier calls from a processing standpoint. With this information in mind, it is best to make wise use of Logon by not needlessly logging on over and over for repetitive functions. For instance, an application that batch processes several bookings could essentially call Logon at the beginning of the process and then Logout at the end after all bookings have been processed.

The explicit use of Logout is also recommended to prevent unexpired (abandoned) sessions from building up and causing application degradation.
Note: Excessive abandoned sessions can result in resource constraints and performance degradation.

Session Pooling

The use of session pooling can reduce the number of individual sessions in memory. An effective session pool limits the number of sessions that can be used concurrently and employs a check-in/checkout system with the sessions. As sessions are freed up they can be checked-in and checked-out by other threads so as to limit the memory footprint on the app server. Sessions can be cleared using the **Clear** method and kept alive using the **KeepAlive** method. When all processing is done the sessions should be logged out, releasing them from memory.

Search requests are stateless calls and do not need to be tied to a specific session. One call after another can be made on the same session without the need to call **Clear** or **Logout**. Reuse of sessions in this case can reduce stress on the system and prevent unnecessary session usage.

Session Timeout

New Skies allows the session timeout to be defined by role. Care should be taken to determine the appropriate setting for each role so that sessions do not timeout too fast and do not remain idle too long. Applications that employ numerous concurrent logins should be given a short timeout to prevent the buildup of unintentionally abandoned sessions.

As a general rule session timeout should be between 15-20 mins for web users, 10 mins for anonymous users, and 5-10 mins for batch processes. It is recommended to employ separate roles in order to take advantage of these restrictions.

HTTP Compression

The New Skies web servers have been configured to return a compressed response when requested via the HTTP header. To ensure the highest performance, compression should be enabled via the request header.

Enabling Compression

Compression can be enabled by referencing the Navitaire.NPS.HTTPCompressionServices.dll assembly available in the API Runner folder. After adding this assembly as a reference in your application, please add the following to you application/web configuration file:

```
<configuration>
<system.net>
<webRequestModules>
<remove prefix=“http:“/>
<add prefix=“http:” type=“Navitaire.NPS.HTTPCompresssionServices.CompressibleHttpRequestCreator,
Navitaire.NPS.HTTPCompresssionServices“ />
<remove prefix=“https:“/>
<add prefix=“https:” type=“Navitaire.NPS.HTTPCompresssionServices.CompressibleHttpRequestCreator,
Navitaire.NPS.HTTPCompresssionServices“ />
</webRequestModules>
</system.net>
</configuration>
```

Example of a header *not* asking for a compressed response:

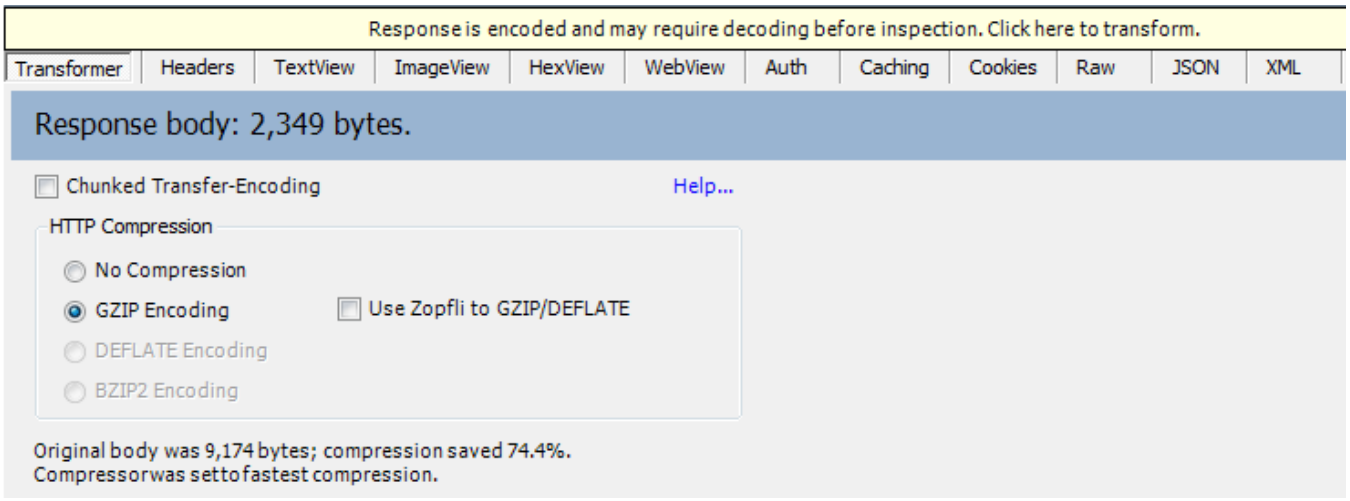
```
Connection: Keep-Alive
Content-Length: 906
Content-Type: text/xml; charset=utf-8
Accept: text/html, */*
Host: 1Lapi.navitaire.com
User-Agent: Java1.6.0_45
SOAPAction: "http://schemas.navitaire.com/WebServices/IBookingManager/GetBooking
```

Example of a header properly asking for a gzip compressed response:

```
Connection: Keep-Alive
Content-Length: 906
Content-Type: text/xml; charset=utf-8
Accept: text/html, */*; q=.2
Accept-Encoding: gzip, deflate
Host: 1Lapi.navitaire.com
User-Agent: Java1.6.0_45
SOAPAction: "http://schemas.navitaire.com/WebServices/IBookingManager/GetBooking
```

Fiddler

Fiddler can be used to determine if the requests you are sending are being returned compressed. Check the response returned to see if the "GZIP Encoding" is selected:



Filtered Methods

In an effort to reduce that amount of data returned in the XML response messages a number of filtered methods have been introduced. Where a filtered method is available these are considered the recommended call to use over the standard (non-filtered) method.

BookingManager

Available filtered methods:

- **GetBookingFromStateFiltered** – replaces the GetBookingFromState method for returning specific components of the current session state booking.

  Note: This method does not return BookingSum, OperationsInfo, LegInfo, or LegSSR collections. If this data is needed, the standard GetBookingFromState method will need to be called.

- **GetBookingStatelessFiltered** – this is the only method to retrieve a booking and not load it into session state.

OperationManager

Available filtered methods:

- **GetManifestFiltered** – replaces the GetManifest method. This method should be used for all manifest calls as the GetManifest method is considered obsolete.

  Note: Only properties that are needed should be set to true. The IncludePassengerSecurityInfo should only be requested if security info is needed as this is calculated and takes additional processing time.

Availability & Booking

Availability Methods and Usage

The availability methods are the most extensively used methods in the API and proper use of these methods protects the overall API environment. Please follow the recommendations below to select not only the correct availability method for the use case but also the appropriate criteria to include in your availability request.

General Recommendations

- Only request the data that you *need* (note "need", not "wouldn't it be nice if…"). For shopping flows, the usual recommendation is a 7-day low-fare request *plus* a 1-day full availability request (GetAvailability) for the selected outbound and return days.
- Don't use GetTripAvailabilityWithSSR unless there is truly a *need* for immediate SSR pricing across all flights and fares in the availability response (this should be very rare – note "need", not "want").
- Wherever possible, use GetAvailability to get flights and fares, and then make a separate call to GetSSRAvailabilityForBooking after selecting and 'selling' the desired flights and fares; for example, when moving to the next page of the shopping flow.
- We should not discuss or recommend GetTripAvailabilityWithSSR with callers unless *we* believe that there is a true *need* that has been identified. There are no defined SLAs for GetTripAvailabilityWithSSR.

Availability Types and Considerations

The follow section describes the availability methods exposed via the API and the recommended usage.

GetAvailability

- The GetAvailability method returns scheduled service and fare availability data for **one** market. Includes all fares for all flights unless LowFare flag is set to true.
- If LowFare flag is set to true a 31-day limit is imposed.
- If LowFare flag is set to false (and usually it is), then the allowed number of days is limited only by the Booking – Flight Search > Maximum Date Range in Availability role setting.
- Can be exponentially heavy hit on the OLTP database, as can GetLowFareTripAvailability. With GetAvailability the response sizes are significant which requires greater network time. With GetLowFareTripAvailability the response sizes are smaller, but the other backend work is similar.

GetLowFareAvailability

- Gets flight availability and fare information for **one** market for an extended period of time – and returns summary information on the lowest flight/fare on that day in the market. This returns one flight per day per market and the lowest fare per flight.
- No date-range limit is inherent in the call.

- Does not make use of low fare cache.
- Deprecated as of 3.4.5. Use **GetLowFareTripAvailability** instead.

GetLowFareTripAvailability

- Retrieves flight availability and fare information for **single or multiple markets** for the given period of time and returns summary information on the **lowest fare for each day in the date range** passed in for the market. This will return the **lowest available fare per day per market**.
- Used to retrieve targeted results for a specific trip type, nights-stay counts, and so on.
- Typically 31-day load factor.
- Makes use of low fare cache when enabled in the environment.

GetTripAvailabilityWithSSR

- Retrieves flight availability and fare information with a list of SSR prices per segment/fare/pax type/program level.
- Functionally, this method is similar to the GetAvailability method with the addition of SSR pricing and the SSR code to include in the response.
- Wherever possible, use GetAvailability to get flights and fares, and then make a separate call to GetSSRAvailabilityForBooking after selecting and 'selling' the desired flights and fares; for example, when moving to the next page of the shopping flow.

Create Booking Flow

When creating bookings using the 3.4.x New Skies API, the following flow and suggestions should be followed.

Availability

Getting availability is recommended so that the correct fares can be sold. You are able to request availability for multiple flights in the same request. Please follow the guidelines in the availability usage section of this document.

GetAvailability

The **GetAvailability** call should ***only*** be used to retrieve a specific day's availability or a small date range (3-5 days). This is an expensive call and each day/O&D requested is counted as 1 look on your look-to-book report.

FareClassControl Filter

The "FareClassControl:Default" will return every fare for every flight that the logged in role can see. This is the largest and most resource intensive of the FareClassControl options. This call should be used extremely rarely and only if the specific case that all fares are needed for display or processing.

The "FareClassControl:LowestFareClass" will return only the lowest fare for each flight. This is the most common case and should be used as the default in most applications.

The "FareClassControl:CompressByProductClass" will return the lowest fare for each ProductClass specified in the availability request. This option should be used when fares from multiple product classes are desired.

The size of the response increases based on the number of days and fares included. Large responses can take additional time to transmit. Excessive and improper availability requests are a leading cause of poor system performance.

GetLowFareTripAvailability

The **GetLowFareTripAvailability** call should be used to retrieve multiple days' worth of availability. This call only charges 1 look no matter how many days are included. This call is good for populating calendars and week views. For each day included in the request only the lowest fare for each flight will be returned. The GetAvailability call would subsequently be used once a specific date is selected to return the full availability for that day and obtain JourneySellKey and FareSellKey for selling the flight.

Price Itinerary

The GetItineraryPrice method is designed to provide a full quote of flights and services prior to selling. The response allows the client to see the full breakdown of fares, fees, charges, and taxes for display.

Sell (Journey)

The recommended method is the SellByJourneyKey option. For this you need to retrieve the JourneySellKey and the FareSellKey from the GetAvailability response for the flights you wish to sell. You are able to sell multiple flights in the same request.

ActionStatusCode Property

The ActionStatusCode property controls how inventory is held in New Skies. For instance, the NN (Need Need) action code tells New Skies to confirm the space immediately thus blocking any other user from acquiring this inventory as long as the session is active. This can be useful for direct booking applications that are converting each sell into a confirmed booking.

On the other hand, for shopping applications (i.e. API driven websites) the recommendation is to use the NU action code which is an unconfirmed hold of the inventory. This hold results in less contention on the New Skies system and prevents massive amounts of requests (perhaps from a scraper or bot) from blocking all the inventory.

Note: Unless inventory is required to be 100% guaranteed at the time of sell (use discernment) it is recommended to use the ActionStatusCode **NU** in your Sell request.

UpdatePassengers

This method is intended to be used to update/replace all passenger information for the designated passenger number. You can update multiple passengers in the same request.

The following collection in the UpdatePassengers request are considered obsolete and should not be populated:

- **PassengerInfants** – the Infant singleton should be used instead.
- **PassengerInfos** – the PassengerInfo singleton should be used instead.
- **PassengerTypeInfos** – the PassengerTypeInfo singleton should be used instead.
- **PsuedoPassenger** – this property is no longer used and will be ignored.

Note: Data sent in this request for each passenger overwrites any previous data for that passenger. Any previous passenger data should be copied over to retain it.

AssignSeats

A single request can be used to assign seats on multiple segments. Refer to the Web Services Developer Guide for detailed information on properly using this method.

Sell (SSR)

This is another option of the Sell method. It is possible to sell multiple SSRs in the same request.

UpdateContacts

This method is intended to be used to add contact information to the booking. This was previously done during Commit, but has been broken out into its own method. This is available in 3.4.5 and higher.

AddPaymentToBooking

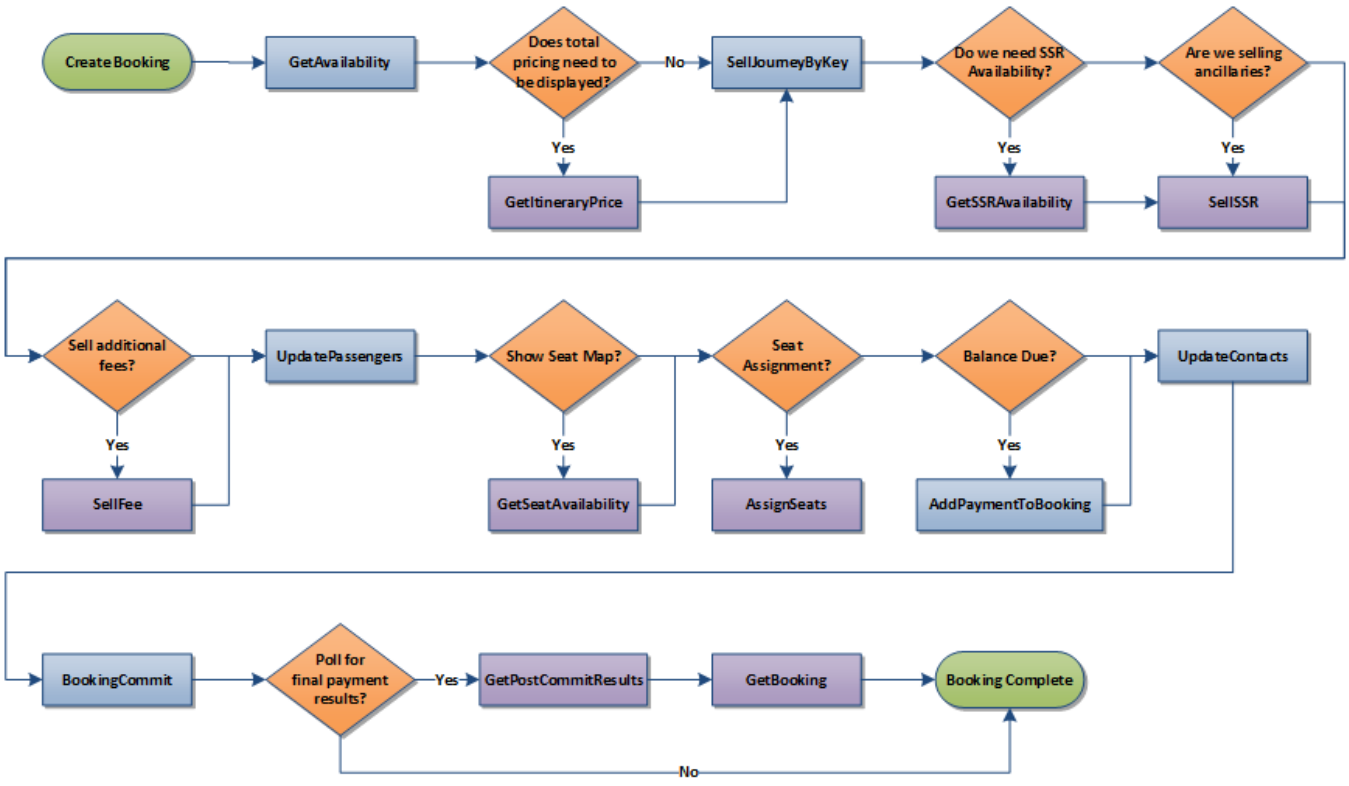
This method is used for the sole purpose of adding payment to the booking in session state.

BookingCommit

Calling this method with an empty request will commit the booking in session state to the database. Additional optional collections (booking contacts, booking comments, SourcePOS, and 3rd party record locators) can be added to the request to be submitted with the booking.

Sample Booking Process Flow

The diagram below shows a recommended create booking flow with both required and optional steps.



Use this information along with the API Usage and Call Volume topic in the ***SOAP API Web Services Programming Reference Guide as well as the README file located in the Web Services folders on your customer care site for additional information on correct usage. Web Services documentation on your customer care site is located here:***